

Introduction to Keypoint-MoSeq	1
I. Understanding the AR-HMM	1
II. Keypoint-MoSeq	2
III. Interpreting MoSeq Analysis	3
PCA Analysis	3
Fitting Progress	4
Syllable Similarity Dendrogram	5
Bigram Transition Probabilities	5
References	7

Introduction to Keypoint-MoSeq

Objectives of this document:

- Provide a brief and accessible explanation of how KPMS works
- Explain some important but unintuitive visual KPMS outputs

I. Understanding the AR-HMM

MoSeq utilizes a **Hidden Markov Model (HMM)**. An HMM observes a sequence of outputs that are generated from a sequence of unknown states. The goal of the HMM is to recover the sequence of states that generated the outputs¹.

An HMM seeks to answer the following questions:

- Given a sequence of outputs, what is the most likely sequence of underlying states?
- Given a sequence of outputs, can you estimate the transition and output probabilities?
- What is the probability of observing a certain sequence of outputs?
- What is the probability that the model is in a particular state at any point in the sequence?

Specifically, MoSeq uses an **autoregressive-HMM (AR-HMM)**.

"Autoregressive" refers to each behavioral module being modeled as a **vector autoregressive (VAR)** process². VAR models capture how multiple values change over time. Each variable is modeled as a function of its previous, or "lagged," values and as a function of the lagged values of other variables in the system³.

The short timescale pose dynamics of behavior are reflected by VAR processes. The longer timescale relationships between behavioral modules are governed by the transition probabilities specified by the HMM². "The AR-HMM proposes that mouse behavior is built from modules, each of which encodes an autoregressive trajectory through pose space, and which transition from one another with definable transition statistics."² These modules are often referred to as "syllables". The short timescale pose dynamics are often discussed in terms of "trajectories".

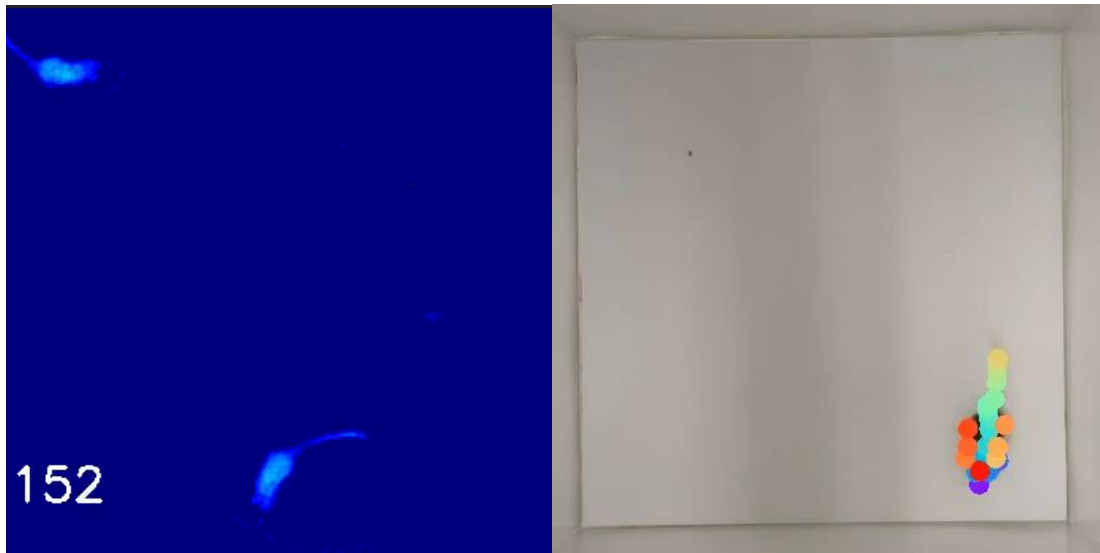
In terms of mouse-behavior, what are the questions the MoSeq AR-HMM seeks to answer?

- Given a sequence of trajectories, what is the most likely sequence of syllables?
- Given a sequence of trajectories can you estimate the transition and trajectory probabilities?
- What is the probability of observing a certain sequence of trajectories?
- What is the probability that the mouse is expressing a particular syllable at any point in the sequence.

II. Keypoint-MoSeq

Keypoint-MoSeq (KPMS) differs from MoSeq in terms of initial input. MoSeq takes depth recordings as its input, while KPMS takes "keypoints." Depth recordings use depth sensors to collect the position of the entire mouse in each frame of video. Keypoint tracking uses supervised learning to track user-defined points on the body of a mouse in each frame of video.

Figure 1. Depth video (left) and keypoint tracking (right)



What is important to consider when working with keypoint data instead of depth data?

1. Keypoint selection
2. Tracking errors and noise

When working with keypoints, it is up to the researcher to decide what points should be tracked and used in downstream analysis. An input of depth data allows downstream algorithms to consider the entire mouse, but with keypoints, a decision has to be made about what points are essential for describing pose-dynamics.

Tracking errors and noise were challenging issues to contend with in the development of KPMS. MoSeq is designed to treat input data as an accurate representation of an animal's position, but keypoint data is composed of noisy observations. MoSeq performed poorly with keypoint data, because it confused "jitter" and tracking errors with real behaviors. KPMS addresses this using **switching linear dynamical systems (SLDS)**. In brief, the SLDS model estimates the animal's location and pose for each frame as well as identity and content of the current behavioral module.

III. Interpreting MoSeq Analysis

PCA Analysis

When working with machine learning models, it is useful to reduce the dimensionality of the input dataset by eliminating redundant or irrelevant features and focusing on the features that contribute the most to variation in the dataset. **Principal Component Analysis (PCA)** expresses data in terms of its **Principal Components (PCs)**. PCs are the directions in the dataset that account for the highest variance and capture the most information⁴. To perform its PCA analysis, KPMS uses `sklearn.decomposition`'s PCA class.

PCA is used to determine the value of the `latent_dimension` parameter in fitting KPMS. This parameter specifies the dimension of the pose trajectory used to fit KPMS. By default, KPMS provides two plots describing the PCA. Figure 2 shows the direction of perturbation (opaque points) from the mean pose (translucent points) for each PC. Figure 3 is a scree plot; it tells us how many PCs are necessary to explain some fraction of the variance.

Figure 2. Principal Components

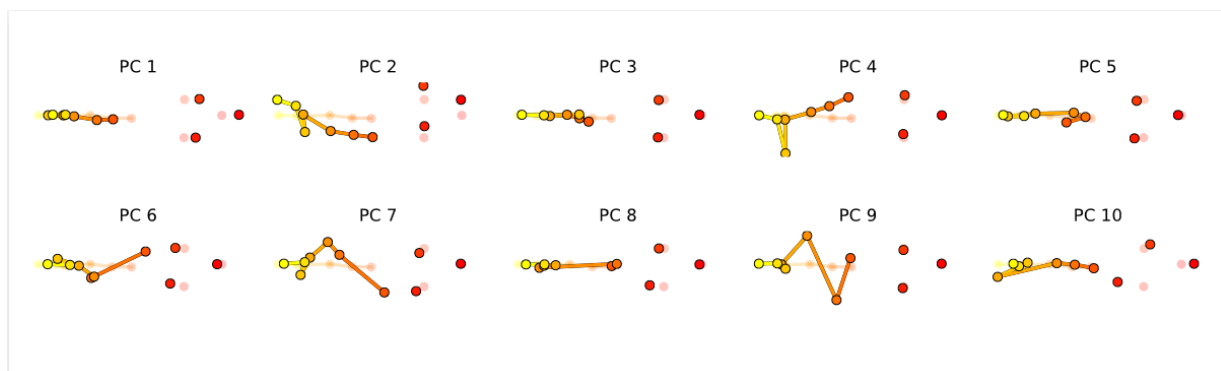
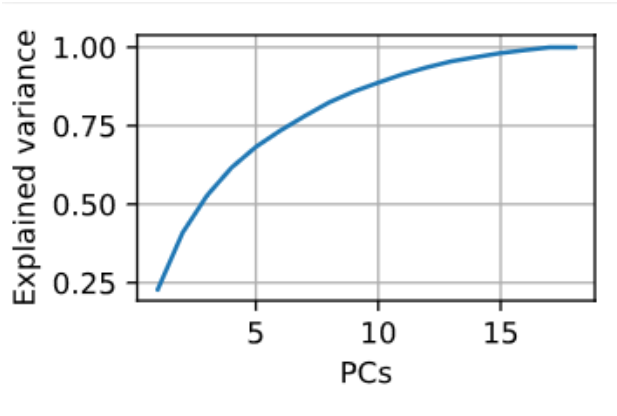


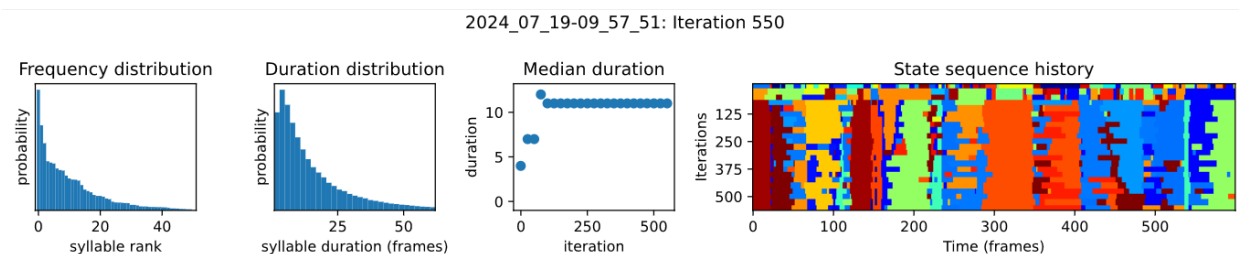
Figure 3. Scree Plot



Fitting Progress

When fitting a model, whether it's the initial AR-HMM or the full model, a set of plots is generated throughout the process (Figure 4). These plots are important for verifying syllable duration. Syllable duration is controlled by the **stickiness hyperparameter (kappa)** which influences the frequency with which syllables can transition⁵. The KPMS team recommends a target duration of 400ms (12 frames at 30 fps).

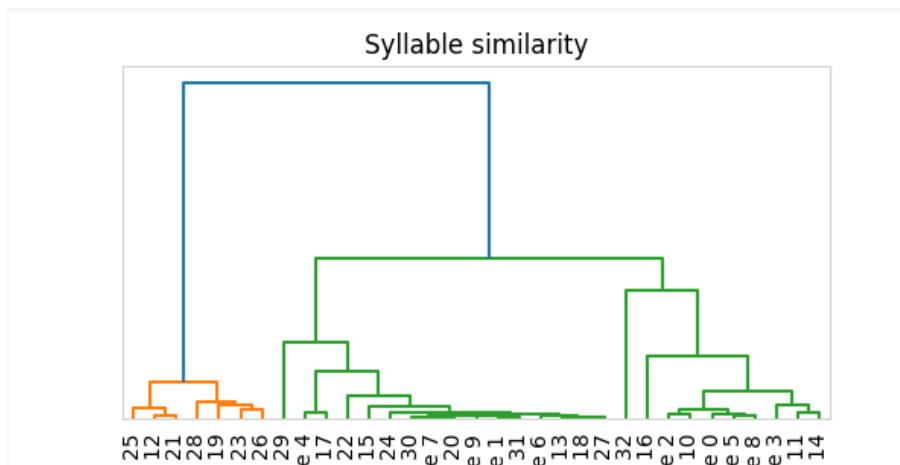
Figure 4. Model Fitting



Syllable Similarity Dendrogram

Dendrograms are used to visualize hierarchical clustering (Figure 5). Think of this like a family tree of syllables; it tells us which ones are "related."

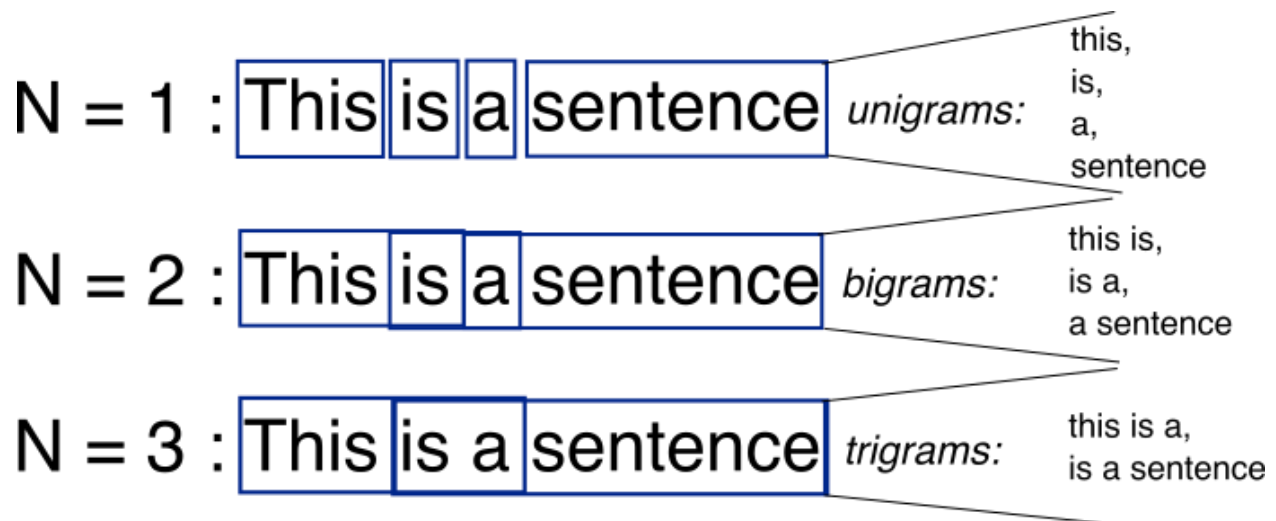
Figure 5. Syllable Similarity Dendrogram



Bigram Transition Probabilities

N-gram analysis is a computational method from the field of **Natural Language Processing (NLP)**. An n-gram is a sequence of words of length n extracted from a larger sequence of words⁶. A unigram uses sequences of one word; a bigram uses sequences of two words.

Figure 6. Types of n-grams



N-gram language models predict the following word in a sequence given the previous $n-1$ words. So, a bigram would predict the next word using the preceding word, and a trigram would predict the next word using the preceding two words⁷. The probability distribution of the n-gram model is based on the Markov Assumption: the probability of a word depends only on the previous $n-1$ words.

MoSeq gives us modules that make up sequences of behavior. We can apply strategies from NLP by treating these modules as words and sequences of behavior as sentences. Bigram Network plots (Figure 7) represent each “word” as a node, and the line between two nodes represents the probability of two words occurring consecutively. Network plots are helpful for visualizing how words are connected, but they do not usually indicate which word occurred before the other, just that they occurred together. Heatmaps (Figure 8) can address this limitation by putting the incoming word on one axis and the outgoing word on the other.

Figure 7. Bigram Network Plots

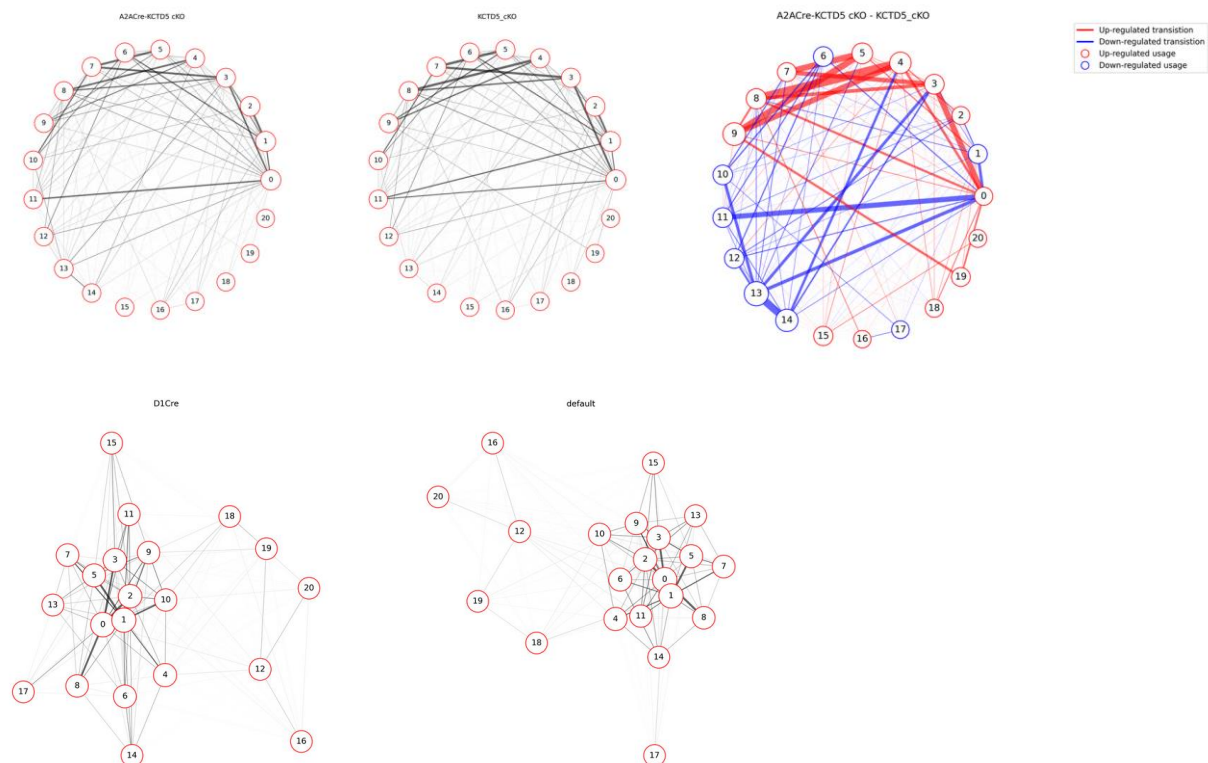
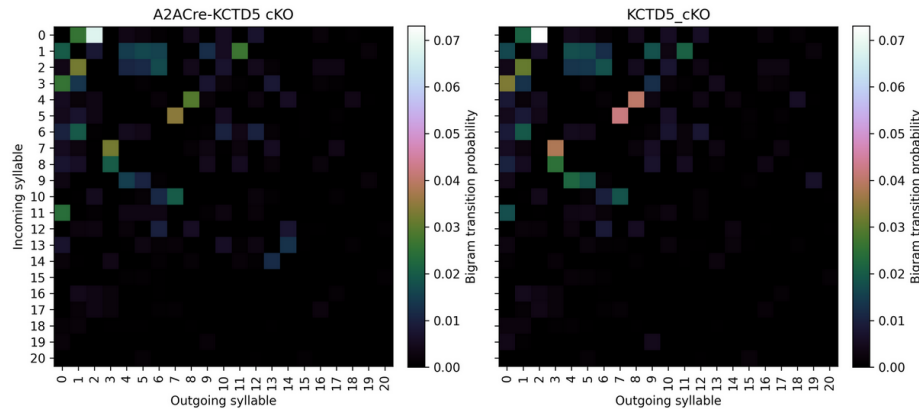


Figure 8. Bigram Transition Heatmap



Why are we focusing on Bigram transition probabilities and not any other type of n-gram? Bigram analysis is the norm in the current literature, and it's the type of n-gram analysis included in the KPMS statistics package. There may be value in trying n-gram analysis with n not equal to two, but I've yet to see anyone do this with MoSeq.

References

[1] "Hidden Markov Models (HMM)." *MathWorks*, The MathWorks, Inc., <https://www.mathworks.com/help/stats/hidden-markov-models-hmm.html>.

[2] Wiltschko, Alexander B., et al. "Mapping Sub-Second Structure in Mouse Behavior." *Neuron*, vol. 88, no. 6, Dec. 2015, pp. 1121-35. *DOI.org (Crossref)*, <https://doi.org/10.1016/j.neuron.2015.11.031>.

[3] "Vector Autoregression (VAR) for Multivariate Time Series." *GeeksforGeeks*, 14 May 2024, <https://www.geeksforgeeks.org/machine-learning/vector-autoregression-var-for-multivariate-time-series/>.

[4] Xiangyu Kong, et al. *Principal Component Analysis Networks and Algorithms*. Springer, 2017. *EBSCOhost*, research.ebsco.com/linkprocessor/plink?id=d7eccc18-020a-3ff0-9176-e7172df27d7b.

[5] Weinreb, Caleb, et al. "Keypoint-MoSeq: Parsing Behavior by Linking Point Tracking to Pose Dynamics." *Nature Methods*, vol. 21, no. 7, July 2024, pp. 1329-39. www.nature.com, <https://doi.org/10.1038/s41592-024-02318-2>.

[6] "An N-Gram Analysis of Communications 2000-2010." *Communications of the ACM*, vol. 55, no. 5, May 2012, pp. 81-87. EBSCOhost, <https://doi.org/10.1145/2160718.2160737>.

[7] "N-Gram Language Modelling with NLTK." *GeeksforGeeks*, 30 May 2021, <https://www.geeksforgeeks.org/nlp/n-gram-language-modelling-with-nltk/>.